

	Computación	Docente: Ing. Patsy Malena Prieto., MSc.
	Programación y Plataformas Web	Período Lectivo: Septiembre 2022 Febrero 2023



FORMATO DE GUÍA DE PRÁCTICA DE LABORATORIO / TALLERES / CENTROS DE SIMULACIÓN – PARA DOCENTES

CARRERA: Ingeniería en Ciencias de la Computación | **ASIGNATURA:** Programación y Plataformas Web

NRO. PRÁCTICA: 11 | **TÍTULO PRÁCTICA:** HttpClient en Angular

OBJETIVO Aplicar HttpClient en Angular

INSTRUCCIONES (Detallar las instrucciones que se dará al estudiante):

1. Entender el uso de HttpClient
2. Usar archivos JSON con métodos GET
3. Crear un reporte usando archivos JSON

ACTIVIDADES POR DESARROLLAR

ESTRUCTURA DE UNA APLICACIÓN WEB USANDO API REST



Métodos HTTP

SAFE METHODS NO ACTION ON SERVER	{	GET	HTTP/1.1 MUST IMPLEMENT THIS METHOD
		HEAD	INSPECT RESOURCE HEADERS
MESSAGE WITH BODY SEND DATA TO SERVER	{	PUT	DEPOSIT DATA ON SERVER – INVERSE OF GET
		POST	SEND INPUT DATA FOR PROCESSING
		PATCH	PARTIALLY MODIFY A RESOURCE
		TRACE	ECHO BACK RECEIVED MESSAGE
		OPTIONS	SERVER CAPABILITIES
		DELETE	DELETE A RESOURCE – NOT GUARANTEED

GET

El método GET se emplea para leer una representación de un **resource**. En caso de respuesta positiva (200 OK), GET devuelve la representación en un formato concreto: HTML, XML, JSON o imágenes, JavaScript, CSS, etc. En caso de respuesta negativa devuelve 404 (*not found*) o 400 (*bad request*).

POST

Aunque se puedan enviar datos a través del método GET, en muchos casos se utiliza POST por las **limitaciones de GET**. En caso de respuesta positiva devuelve 201 (*created*). Los POST requests se envían normalmente con formularios

PUT

Utilizado normalmente para **actualizar contenidos**, pero también pueden **crearlos**. Tampoco muestra ninguna información en la URL. En caso de éxito devuelve 201 (*created*, en caso de que la acción haya creado un elemento) o 204 (*no response*, si el servidor no devuelve ningún contenido). A diferencia de POST es **idempotente**, si se crea o edita un resource con PUT y se hace el mismo request otra vez, el resource todavía está ahí y mantiene el mismo estado que en la primera llamada. Si con una llamada PUT se cambia aunque sea sólo un contador en el resource, la llamada ya no es idempotente, ya que se cambian contenidos.

DELETE

Simplemente elimina un **resource** identificado en la **URI**. Si se elimina correctamente devuelve 200 junto con un *body response*, o 204 sin *body*. DELETE, al igual que PUT y GET, también es **idempotente**.

HEAD

Es idéntico a GET, pero el servidor no devuelve el contenido en el **HTTP response**. Cuando se envía un **HEAD request**, significa que sólo se está interesado en el código de respuesta y los **headers HTTP**, no en el propio

	Computación	Docente: Ing. Patsy Malena Prieto., MSc.
	Programación y Plataformas Web	Período Lectivo: Septiembre 2022 Febrero 2023

documento. Con este método el navegador puede comprobar si un documento se ha modificado, por razones de caching. Puede comprobar también directamente si el archivo existe.

1. Crear un servicio usando la sentencia *ng generate service nombreServicio*

```
C:\PPrieto\Nuevo vol\clases\docs\material clase\programacion y plataformas web\VSCode\practicas\site004>ng g s gasto
CREATE src/app/gasto.service.spec.ts (352 bytes)
CREATE src/app/gasto.service.ts (134 bytes)
```

2. Necesitamos declarar el servicio para que pueda ser usado en la aplicación. Para esto lo importamos en *app.module.ts* añadiendo las siguientes líneas de código:

```
import { HttpClientModule } from '@angular/common/http';
import { GastoService } from './gasto.service';
```

```
imports: [HttpClientModule]
```

```
providers: [GastoService]
```

3. En el archivo *gasto.service.ts* se va a declarar las funciones que permitirán acceder y manipular los datos que se encuentran en formato JSON usando la API Rest. En primer lugar se importa la librería *HttpClient*.

```
import {HttpClient} from '@angular/common/http'
```

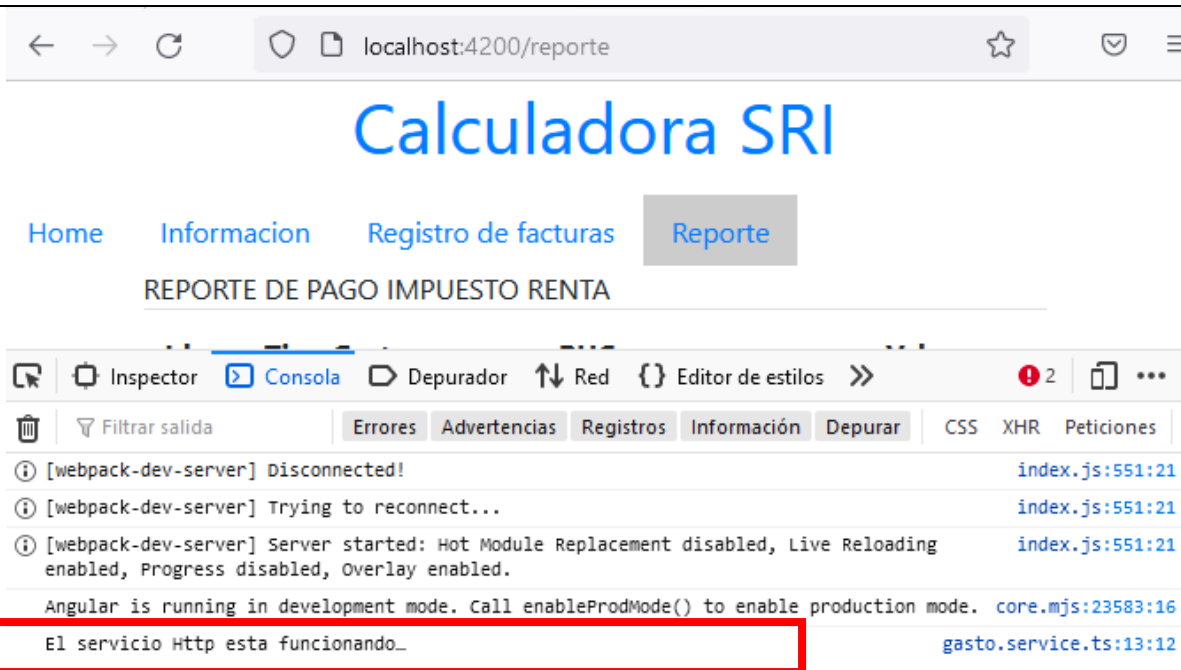
4. Se creará un constructor en el archivo *gasto.service.ts* para verificar que el servicio este funcionando.

```
constructor(private httpClient: HttpClient) {
  console.log('El servicio Http esta funcionando...')
}
```

5. Además el componente donde se usará el servicio es necesario colocar en el constructor del archivo *ts* la siguiente declaración. En el caso de esta práctica corresponde al archivo *reporte.component.ts*

```
import { GastoService } from '../gasto.service';
```

```
constructor(private gastoService:GastoService) {
}
```



- Se crea un archivo JSON denominado datos.json con la siguiente estructura. Este archivo debe colocarse en la carpeta /assets

```
practicas > site004 > src > assets > {} datos.json >
1  [{
2    "id":1,
3    "tipo":"vivienda",
4    "ruc":"171345672001",
5    "valor":345.6
6  },
7  {
8
9    "id":2,
10   "tipo":"salud",
11   "ruc":"171200972001",
12   "valor":45.6
13 },
14 {
15   "id":3,
16   "tipo":"alimentacion",
17   "ruc":"180345672001",
18   "valor":1345.6
19 }]
```

- Para manipular los datos del archivo JSON se recomienda usar un archivo que contenga la estructura. Para esto se creará una interface que permite definir un objeto con los atributos establecidos en el archivo JSON. En la carpeta app se crea un nuevo archivo llamado Gasto.ts. Se declara los elementos con su tipo de dato.

```
practicas > site004 > src > app > TS Gasto.ts > ➔ Gasto >
```

```
1  export interface Gasto{  
2      "id":number;  
3      "tipo":string;  
4      "ruc":string;  
5      "valor":Float32Array;  
6  }
```

8. Se crea un método GET que permita devolver los valores del archivo JSON datos.json. Este método se coloca en `gasto.service.ts`. Este método permite usar el método GET, se define el tipo de dato que devuelve que en este caso es de tipo `Gasto[]`. Además se declara una constante con la URL donde se encuentra alojada el archivo JSON.

```
import { Gasto } from './Gasto';
```

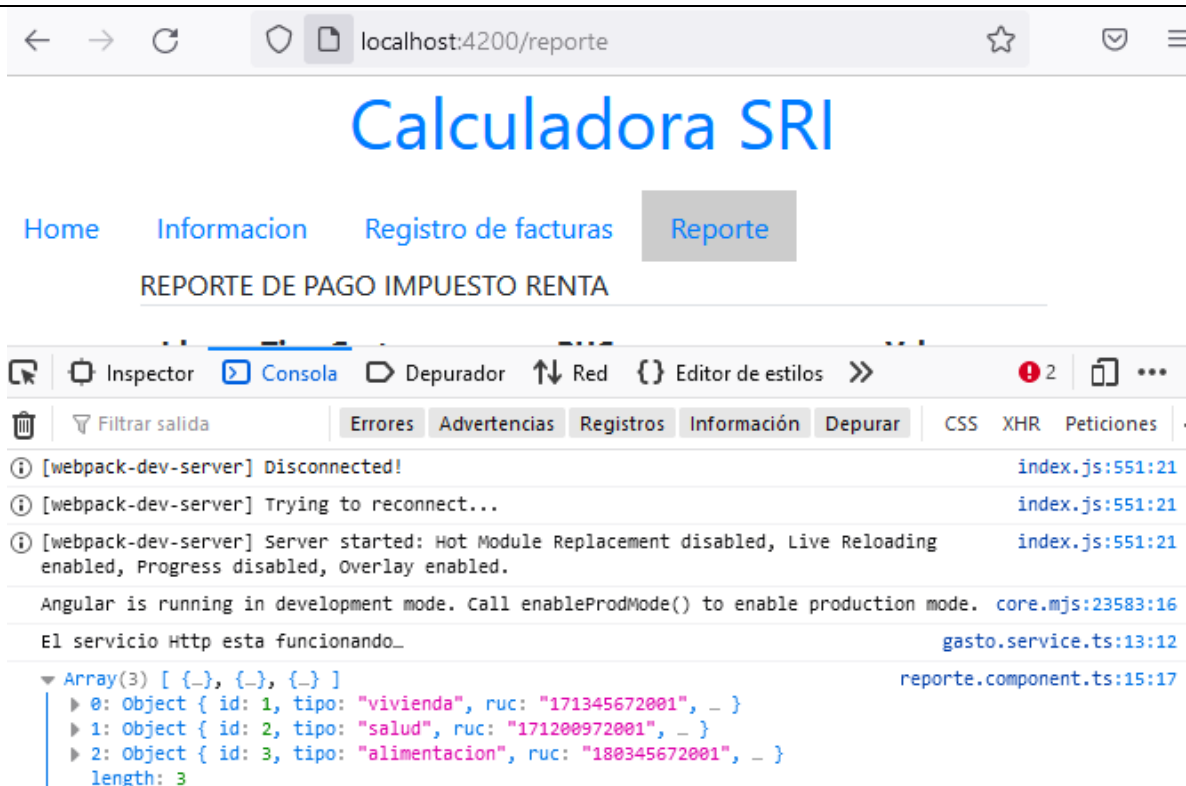
```
const configUrl='assets/datos.json';
```

```
obtenerDatos(){  
    return this.httpClient.get<Gasto[]>(configUrl);  
}
```

9. Desde el componente que se necesite se puede llamar al método GET o a cualquier otro método HTTP que se requiera. En este caso lo utilizamos desde el componente `reporte`. En este caso hemos impreso el contenido el archivo JSON en la consola del navegador.

```
export class ReporteComponent implements OnInit{  
  
    constructor(private gastoService:GastoService) {  
        this.gastoService.obtenerDatos().subscribe(data =>  
        {  
            console.log(data);  
        });  
    }  
}
```

Este método puede estar colocado en el constructor, en ese caso se mostrará directamente cuando se llame a la página o puede colocarse en el método `ngOnInit()` en ese caso primero se cargará la página y de forma posterior se cargarán los datos



10. Para mostrar la información en el componente reporte se usará la interface `Gasto.ts` que es el modelo de JSON. En el componente correspondiente, en este caso en el archivo `reporte.component.ts` seguimos modificando el constructor.

Añadimos una variable `gastos` del tipo `Gasto[]` y la llenamos con la información que se obtiene del servicio `obtenerDatos()`

```
export class ReporteComponent implements OnInit{
  gastos:Gasto[]=[];
  constructor(private gastoService:GastoService) {
    this.gastoService.obtenerDatos().subscribe(data =>
    {
      console.log(data);
      this.gastos=data;
    });
  }
}
```

11. En la interfaz gráfica `reporte.component.html` se usará un componente Table de Bootstrap <https://getbootstrap.com/docs/5.1/content/tables/>

```

practicass > site004 > src > app > reporte > <> reporte.component.html > {
1  <div class="container">
2      REPORTE DE PAGO IMPUESTO RENTA
3      <div >
4          <table class="table">
5              <thead>
6                  <tr>
7                      <th scope="col">Id</th>
8                      <th scope="col">Tipo Gasto</th>
9                      <th scope="col">RUC</th>
10                     <th scope="col">Valor</th>
11                 </tr>
12             </thead>
13             <tbody>
14                 <tr *ngFor="let gasto of gastos">
15                     <th scope="row">{{gasto.id}}</th>
16                     <td>{{gasto.tipo}}</td>
17                     <td>{{gasto.ruc}}</td>
18                     <td>{{gasto.valor}}</td>
19                 </tr>
20             </tbody>
21         </table>
22     </div>
23

```

12. Se incluirá una sentencia *ngFor para recorrer el resultado del servicio *obtenerDatos()*.

```

<tbody>

    <tr *ngFor="let gasto of gastos">
        <th scope="row">{{gasto.id}}</th>
        <td>{{gasto.tipo}}</td>
        <td>{{gasto.ruc}}</td>
        <td>{{gasto.valor}}</td>
    </tr>
</tbody>

```

13. Se mostrará los datos del archivo JSON considerando los atributos colocados en el archivo Gasto.ts

Calculadora SRI

[Home](#) [Informacion](#) [Registro de facturas](#) [Reporte](#)

REPORTE DE PAGO IMPUESTO RENTA

Id	Tipo Gasto	RUC	Valor
1	vivienda	171345672001	345.6
2	salud	171200972001	45.6
3	alimentacion	180345672001	1345.6

14. También puede ser posible que se requiere mostrar los datos de un JSON almacenado en un repositorio externo. En ese caso usaremos los datos publicados en <https://jsonplaceholder.typicode.com/users>. Lo primero es crear una interface User.ts con los datos que vamos a mostrar en la aplicación.

```
practica07 > src > app > TS User.ts > ➔ User
```

```
1 export interface User{
2     "userId":number;
3     "name":string;
4     "username":string;
5     "email":string;
6 }
```

15. Es necesario crear un nuevo servicio en este caso se llamará User. Se creará un método GET con el nombre obtenerDatos(). Al colocar el URL del archivo JSON se coloca directamente el URL donde se encuentra publicado el archivo.

```
import { User } from './User';
```

```
obtenerDatos(){
    return this.httpClient.get<User[]>('https://jsonplaceholder.typicode.com/users');
}
```


localhost:4200/reporte

object Object

Inspector Consola Depurador Red Editor de estilos Rendimiento Memoria Almacenamiento Accesib

Filtrar salida

GET http://localhost:4200/assets/sryi.jpg

Angular is running in development mode. Call enableProdMode() to enable production mode.

El servicio Http esta funcionando

```

Array(10) [ (-), (-), (-), (-), (-), (-), (-), (-), (-), (-) ]
  ▶ 0: Object { id: 1, name: "Leanne Graham", username: "Bret", - }
  ▶ 1: Object { id: 2, name: "Ervin Howell", username: "Antonette", - }
  ▶ 2: Object { id: 3, name: "Clementine Bauch", username: "Samantha", - }
  ▶ 3: Object { id: 4, name: "Patricia Lebsack", username: "Karianne", - }
  ▶ 4: Object { id: 5, name: "Chelsey Dietrich", username: "Kamren", - }
  ▶ 5: Object { id: 6, name: "Mrs. Dennis Schulist", username: "Leopoldo_Corkery", - }
  ▶ 6: Object { id: 7, name: "Kurtis Weissnat", username: "Elwyn.Skiles", - }
  ▶ 7: Object { id: 8, name: "Nicholas Runolfsson", username: "Maxime_Nienow", - }
  ▶ 8: Object { id: 9, name: "Glenna Reichert", username: "Delphine", - }
  ▶ 9: Object { id: 10, name: "Clementine DuBuque", username: "Moriah.Stanton", - }
  length: 10
  <prototype>: Array []
  
```



Trabajo para practicar

16. Crear un componente adicional para mostrar la información de User en una interfaz gráfica.

localhost:4200/reporte

Calculadora del Impuesto a la Renta

Esta aplicación permite el registro y cálculo posterior del impuesto a la renta

Id	Nombre	Login	Correo
1	Leanne Graham	Bret	Sincere@april.biz
2	Ervin Howell	Antonette	Shanna@melissa.tv
3	Clementine Bauch	Samantha	Nathan@yesenia.net
4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org

RESULTADO(S) OBTENIDO(S):

Calculadora SRI

[Home](#) [Informacion](#) [Registro de facturas](#) [Reporte](#)

REPORTE DE PAGO IMPUESTO RENTA

Id	Tipo Gasto	RUC	Valor
1	vivienda	171345672001	345.6
2	salud	171200972001	45.6
3	alimentacion	180345672001	1345.6

CONCLUSIONES:

- El uso de servicios usando métodos HTTP permiten que las aplicaciones web sean interoperables, facilitando la conexión real de datos en la web.
- Los métodos HTTP son servicios rápidos, livianos y fáciles de implementar en comparación a otras soluciones distribuidas como SOAP, RMI, CORBA.

RECOMENDACIONES:

- JSON es un estándar en la industria de desarrollo móvil por lo que es necesario usar este tipo de archivos para implementar soluciones móviles.
- JSON es un archivo de texto que incluye los tipos de datos genéricos interpretados por todos los lenguajes de programación por lo que su uso facilita la conexión con cualquier programa.

Docente / Técnico Docente:

Firma: _____